Week 2, Day 3: @Transactional, Isolation and Intervals

Prepared for Upendra Kumar Sharma | 3–4 hours | Main topic: Spring transactions, propagation, isolation, rollback rules | DSA: Intervals

Today's Schedule

- **0:00–0:40** Study how @Transactional works and the propagation types (sections 1 and 2).
- **0:40–1:20** Study isolation levels and rollback rules (sections 3 and 4).
- **1:20–1:50** Study the pitfalls (section 5), then reproduce at least three of them in a small Spring Boot project with PostgreSQL.
- **1:50–2:50** DSA: solve the 3 interval problems (section 7). Spend at most 25 minutes per problem before reading the hint.
- **2:50–3:20** Answer the interview questions aloud (section 8) and add 10 flashcards.

1. How @Transactional Works

- **Transaction basics (ACID):** Atomicity (all or nothing), Consistency (rules stay valid), Isolation (concurrent transactions do not corrupt each other), Durability (committed data survives crashes).
- **Declarative transactions:** you annotate a method or class, and Spring handles begin, commit and rollback for you.
- **Mechanism:** the bean is wrapped in an AOP proxy (Day 2). When a call comes through the proxy, a **TransactionInterceptor** asks the **PlatformTransactionManager** to start or join a transaction, runs your method, then commits, or rolls back if the exception rules say so.
- **Transaction managers:** JpaTransactionManager for JPA and Hibernate, DataSourceTransactionManager for plain JDBC. Spring Boot auto-configures the right one. With several data sources, name the manager: @Transactional("ordersTxManager").
- **The transaction is bound to the current thread** (through thread-local storage), together with its database connection. Code running on a different thread does not take part in it.

- **Which annotation:** use org.springframework.transaction.annotation.Transactional. Spring also understands jakarta.transaction.Transactional, but the Spring one has more attributes.
- **Where to put it:** on the **service layer**, where one method represents one business operation. Class-level @Transactional applies to all public methods, and a method-level one overrides it.
- **Programmatic alternative:** TransactionTemplate, useful when you need a transaction around only part of a method, or want to avoid proxy limits.

```
@Service
public class TransferService {
    private final AccountRepository accounts;
    public TransferService(AccountRepository accounts) { this.accounts =
accounts; }

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accounts.findById(fromId).orElseThrow();
        Account to = accounts.findById(toId).orElseThrow();
        from.debit(amount);          // dirty checking writes the changes
        to.credit(amount);           // when the transaction commits
    }
}
```

2. Propagation: What Happens When a Transactional Method Calls Another

Propagation answers: if a transaction already exists, join it, suspend it, or refuse?

- **REQUIRED (default):** join the current transaction, or create one if none exists. Both methods commit or roll back together.
- **REQUIRES_NEW:** always start a new, independent transaction and **suspend** the current one. The inner transaction commits or rolls back on its own. Typical use: writing an audit log that must be saved even if the main operation fails.
- **SUPPORTS:** join one if it exists, otherwise run without a transaction.
- **NOT_SUPPORTED:** suspend the current transaction and run without one.
- **MANDATORY:** must run inside an existing transaction, otherwise throws an exception.
- **NEVER:** must run without a transaction, throws an exception if one exists.

- **NESTED:** runs inside the current transaction using a **savepoint**, so the inner part can roll back without rolling back the outer one. It relies on JDBC savepoints, and support depends on the transaction manager and JPA dialect, so verify it before depending on it.

Key facts about REQUIRES_NEW

- It needs a **second database connection** while the first is suspended. Overuse under load can exhaust the connection pool, and in the worst case deadlock when every thread holds one connection and waits for another.
- The outer transaction cannot see uncommitted data of the inner one, and vice versa.

```
@Service
public class OrderService {
    private final AuditService audit;
    private final OrderRepository orders;
    // constructor omitted

    @Transactional
    public void placeOrder(Order order) {
        orders.save(order);
        audit.record("ORDER_ATTEMPT", order.getId());  // survives a later
rollback
        validateStock(order);                          // may throw and roll
back the order
    }
}

@Service
class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(String event, Long refId) { /* insert audit row */ }
}
```

Note that AuditService is a **separate bean**. If record() were in the same class and called directly, the proxy would be bypassed (see pitfalls).

3. Isolation Levels

Problems that concurrency can cause

- **Dirty read:** you read data another transaction has not committed yet.

- **Non-repeatable read:** you read the same row twice and get different values, because another transaction committed a change in between.
- **Phantom read:** you run the same query twice and the set of matching rows changes, because another transaction inserted or deleted rows.

Levels and what they prevent

- **READ_UNCOMMITTED:** prevents nothing. Dirty reads are possible.
- **READ_COMMITTED:** prevents dirty reads. Non-repeatable and phantom reads can still happen.
- **REPEATABLE_READ:** prevents dirty and non-repeatable reads. Phantoms are allowed by the SQL standard, though some databases prevent them too.
- **SERIALIZABLE:** prevents all three. Transactions behave as if run one after another. Lowest concurrency and highest cost.
- **DEFAULT:** use the database's default level.

Database specifics (you use PostgreSQL)

- PostgreSQL's default is **READ COMMITTED**. MySQL InnoDB's default is REPEATABLE READ.
- PostgreSQL treats READ UNCOMMITTED the same as READ COMMITTED, so dirty reads never happen there.
- PostgreSQL's REPEATABLE READ uses snapshot isolation and also prevents phantoms.
- PostgreSQL SERIALIZABLE can abort a transaction with a serialization failure. Your code must be ready to **retry**.

Practical guidance

- Stay on the default for most operations. Raise isolation only for a specific operation that needs it, and combine it with locking where necessary.
- For "read, then update based on what I read" logic, prefer **optimistic locking** (a @Version column) or **pessimistic locking** (SELECT ... FOR UPDATE), which Day 5 covers.
- Isolation applies only when a **new** transaction starts. A method joining an existing transaction (REQUIRED) cannot change its isolation level.

```
@Transactional(isolation = Isolation.REPEATABLE_READ, readOnly = true)
public MonthlyReport buildReport(YearMonth month) { /* consistent snapshot */
}
```

4. Rollback Rules and Other Attributes

Default rollback behavior

- Rolls back on **RuntimeException** (unchecked) and **Error**.
- Does **not** roll back on checked exceptions (such as IOException or a custom exception extending Exception). The transaction commits.
- Change this with **rollbackFor = Exception.class**, or exclude specific exceptions with **noRollbackFor**.

The rollback-only trap

If an inner method joined the outer transaction (REQUIRED) and throws a RuntimeException, the transaction is marked **rollback-only**. If the outer method catches that exception and carries on, the commit still fails with **UnexpectedRollbackException**. To let the inner part fail independently, use REQUIRES_NEW or NESTED, or let the exception propagate.

Other attributes

- **readOnly = true:** a hint that allows optimizations. With Hibernate, it skips dirty checking and flushing, which saves time for query-only methods. It is not a guarantee that the database blocks writes.
- **timeout:** seconds before the transaction is marked for rollback.
- **transactionManager / value:** which manager to use when there are several.

```
@Transactional(rollbackFor = Exception.class)
public void importFile(Path file) throws IOException {
    // a checked IOException now rolls the transaction back
}
```

5. Common Pitfalls (Reproduce These Yourself)

1. **Self-invocation:** calling a @Transactional method from another method of the same class bypasses the proxy, so no transaction starts. Move the method to another bean.
2. **Non-public methods and final classes:** proxy-based advice is only reliable on public methods, and CGLIB cannot override final methods or classes.
3. **Checked exceptions do not roll back** unless you configure rollbackFor.
4. **Swallowing exceptions:** a try/catch inside the method that hides the exception makes Spring see a normal return, so it commits.

5. **Transaction too wide:** calling external HTTP services, sending emails or waiting for user input inside a transaction holds a database connection for a long time. Keep transactions short and do slow work outside them.
6. **LazyInitializationException:** reading a lazy relationship after the transaction (and session) has closed. Fetch what you need inside the service, using join fetch, an entity graph or a DTO projection. Spring Boot enables **open-in-view** by default, which hides the problem by keeping the session open during web requests, but it can cause extra queries and long connection use, so many teams turn it off.
7. **New threads and @Async:** the transaction is thread-bound, so work on another thread does not take part in it.
8. **Sending messages before commit:** publishing a Kafka event inside the transaction can announce data that later rolls back. Publish after commit with @TransactionalEventListener(phase = AFTER_COMMIT), or use the **outbox pattern** (Week 3).
9. **Overusing REQUIRES_NEW:** can exhaust the connection pool.
10. **Wrong transaction manager** when the app has multiple data sources.

```
@Component
class OrderEvents {
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void onOrderPlaced(OrderPlacedEvent event) {
        // safe to publish to Kafka now: the order is committed
    }
}
```

Debugging tip: set logging.level.org.springframework.transaction.interceptor=TRACE (and JpaTransactionManager to DEBUG) to see exactly when transactions start, join and commit.

6. Connect This to Your Own Work

Your resume mentions workflows with multi-step processing (applications, document verification, payments, certificate generation). Prepare a short story using this structure: which operation needed to be atomic, what went wrong or could go wrong (partial saves, a failure after a payment step, a certificate generated for a rolled-back application), and which transaction setting or design you used to fix it. Only describe what you actually did.

7. DSA: Intervals (3 Problems)

Pattern: most interval problems start with **sorting** (by start for merging, by end for greedy selection) and then a single scan. Always clarify whether touching intervals such as [1,2] and [2,3] count as overlapping.

Problem 1: Merge Intervals

- **Idea:** sort by start. Keep a result list. If the current interval starts after the last result's end, add it, otherwise extend the last result's end with the maximum.
- **Complexity:** O(n log n) time (sorting), O(n) space for the output.
- **Edge cases:** one interval, fully nested intervals ([1,10] and [2,3]), touching intervals ([1,4] and [4,5] merge).

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> result = new ArrayList<>();
    for (int[] cur : intervals) {
        if (result.isEmpty() || result.get(result.size() - 1)[1] < cur[0]) {
            result.add(cur);
        } else {
            int[] last = result.get(result.size() - 1);
            last[1] = Math.max(last[1], cur[1]);
        }
    }
    return result.toArray(new int[0][]);
}
```

Problem 2: Insert Interval

- **Idea:** the input is already sorted, so scan once in three phases. Add all intervals that end before the new one starts. Merge every interval that overlaps the new one, widening the new interval. Add the merged interval, then the remaining intervals.
- **Complexity:** O(n) time, O(n) space for the output.
- **Edge cases:** empty input, new interval before everything or after everything, new interval swallowing all existing ones.

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < newInterval[0])
result.add(intervals[i++]);
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.add(newInterval);
    while (i < n) result.add(intervals[i++]);
    return result.toArray(new int[0][]);
}
```

Problem 3: Non-overlapping Intervals

- **Idea:** greedy. Sort by **end** time. Keep the interval that ends earliest, since it leaves the most room for the rest. Whenever the next interval starts before the current end, it overlaps and must be removed (count it). Otherwise it becomes the new current interval.
- **Complexity:** O(n log n) time, O(1) extra space.
- **Edge cases:** empty input, intervals that only touch ([1,2] and [2,3] do not overlap), identical intervals.
- **Same pattern as:** activity selection and meeting-room style problems.

```
public int eraseOverlapIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[1], b[1]));
    int end = intervals[0][1], removed = 0;
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] < end) removed++;       // overlap: drop this one
        else end = intervals[i][1];                 // keep it
    }
    return removed;
}
```

8. Interview Questions to Answer Aloud

1. How does @Transactional work internally?
2. Why does a @Transactional method called from another method in the same class not run in a transaction?
3. List the propagation types. What is the difference between REQUIRED and REQUIRES_NEW?
4. Give a real use case for REQUIRES_NEW, and name its risk.
5. What is NESTED propagation, and when does it work?
6. What are dirty, non-repeatable and phantom reads? Which isolation level prevents which?
7. What is the default isolation level in PostgreSQL, and how does it differ from MySQL?
8. Which exceptions cause a rollback by default? How do you change that?
9. What is UnexpectedRollbackException, and what causes it?
10. What does readOnly = true actually do?
11. What causes LazyInitializationException, and how do you fix it properly?
12. How do you make sure a Kafka message is only sent after the database commit?
13. Why should you avoid external calls inside a transaction?

9. End-of-Day Checklist

- [ ] I can explain the proxy mechanism behind @Transactional and name five pitfalls.
- [ ] I can describe all propagation types and give examples for REQUIRED, REQUIRES_NEW and MANDATORY.
- [ ] I can state which anomaly each isolation level prevents and the PostgreSQL defaults.
- [ ] I can explain the default rollback rules and the rollback-only trap.
- [ ] I reproduced self-invocation, checked-exception and lazy-loading problems in a real project.
- [ ] I solved all 3 interval problems and can say why each sorts the way it does.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 2, Day 4): JPA and Hibernate: the N+1 problem, lazy vs eager loading, entity states and caching, plus linked list problems.